

Teste Técnico — Desenvolvedor Backend

Entrega

Submeta o código-fonte em um repositório Git público com instruções de execução local no `README.md`.

Objetivo

Desenvolver uma API utilizando **Node.js** com **TypeScript**, **Serverless Framework** e **AWS Lambda**. A API terá dois endpoints:

1. **Buscar agendas e horários dos médicos.**
2. **Registrar o agendamento de um paciente.**

Os dados retornados deverão ser **mockados** (sem integração com banco de dados real). O foco é avaliar capacidade técnica, domínio de boas práticas e uso correto das tecnologias.

Instruções

- Utilize **Node.js v20** ou superior e **TypeScript**.
 - A API deve ser criada com o **Serverless Framework** e o plugin **serverless-offline** para execução local.
 - As funções devem ser implementadas como **AWS Lambda** com triggers via **AWS API Gateway (REST)**.
 - As respostas dos endpoints devem ser mockadas com dados fictícios, mas em formato realista.
 - Entregue o código em repositório Git público com instruções no README para rodar e, se possível, fazer deploy.
-

Descrição da API

1. Buscar agendas e horários dos médicos

Rota: `GET /agendas`

Descrição: Retorna uma lista de médicos com suas respectivas agendas e horários disponíveis.

Resposta esperada:

```
{
  "medicos": [
    {
      "id": 1,
      "nome": "Dr. João Silva",
      "especialidade": "Cardiologista",
      "horarios_disponiveis": [
        "2026-06-10 09:00",
        "2026-06-10 10:00",
        "2026-06-10 11:00"
      ]
    },
    {
      "id": 2,
      "nome": "Dra. Maria Souza",
      "especialidade": "Dermatologista",
      "horarios_disponiveis": [
        "2026-06-11 14:00",
        "2026-06-11 15:00"
      ]
    }
  ]
}
```

2. Registrar agendamento do paciente

Rota: POST /agendamento

Descrição: Permite que um paciente marque um horário de consulta com um médico.

Payload:

```
{
  "agendamento": {
    "medico_id": 1,
    "paciente": "Carlos Almeida",
    "data_horario": "2026-06-10 09:00"
  }
}
```

Resposta esperada (201):

```
{
  "mensagem": "Agendamento realizado com sucesso",
  "agendamento": {
    "id": "uuid-gerado",
    "medico": "Dr. João Silva",
    "paciente": "Carlos Almeida",
    "data_horario": "2026-06-10 09:00"
  }
}
```

Resposta esperada para horário já ocupado (409):

```
{
  "erro": "Horário indisponível",
  "mensagem": "O horário solicitado não está mais disponível para este médico."
}
```

O controle de conflito pode ser feito em memória com os dados mockados — não é necessário persistência.

Requisitos Técnicos

Funcionais

- Endpoints REST implementados como funções Lambda.
- Respostas mockadas com dados fictícios, sem banco de dados real.
- Validação de payload nas requisições, com retorno de erros adequados (ex: `400` para payload inválido, `409` para conflito de horário).
- Boas práticas de design de APIs RESTful.
- Testes unitários para a lógica de negócio com **Jest**.

Não-funcionais

- **TypeScript** em todo o projeto, com tipagem explícita e sem uso de `any`.
- **README** claro com instruções de execução local e deploy.
- **ESLint** e **Prettier** configurados.

Arquitetura e Princípios

O teste não impõe uma estrutura de pastas. A organização do código é parte da avaliação. Espera-se que o candidato tome decisões arquiteturais conscientes e seja capaz de justificá-las.

Alguns princípios e padrões que serão valorizados:

- **Separação de responsabilidades** — a lógica de negócio não deve residir no handler Lambda. Handlers devem ser finos, delegando para camadas de serviço ou casos de uso.
- **Inversão de dependência** — dependências concretas (ex: repositório de dados, serviço externo) devem ser injetadas, não instanciadas diretamente nas camadas de negócio. Isso torna o código testável sem mocks globais.
- **Princípios SOLID** — espera-se que as decisões de design reflitam consciência sobre os cinco princípios, mesmo que nem todos se apliquem ao escopo do teste.

- **Tratamento de erros como fluxo explícito** — erros de negócio (ex: horário indisponível) não devem ser tratados como exceções genéricas. Prefira classes de erro tipadas com semântica clara.

Diferenciais

Serão vistos com bons olhos, mas **não são obrigatórios**:

- **Testes de integração ou end-to-end** além dos unitários.
- **Decorators** para encapsular funcionalidades transversais (ex: validação de payload, logging, tratamento de erros HTTP).
- **Agente de IA integrado ao fluxo**: implementar um endpoint `POST /triagem` que receba a descrição dos sintomas do paciente em texto livre e retorne uma sugestão de especialidade médica. A integração pode ser feita com qualquer LLM via API (OpenAI, Anthropic, etc.). Será avaliada a qualidade do prompt, o tratamento de falhas na chamada externa e a separação entre a lógica de negócio e a chamada ao modelo.

CrITÉrios de Avaliação

Critério	O que será avaliado
Qualidade do código	Estruturação, modularidade, legibilidade e aderência a boas práticas
Domínio de TypeScript	Tipagem correta, uso de interfaces, generics, ausência de <code>any</code>
Conhecimento de Serverless	Configuração e implementação de funções Lambda com Serverless Framework
Design de API	Rotas bem definidas, status codes corretos, tratamento de erros
Arquitetura	Separação de responsabilidades, decisões justificáveis, coesão
Testes	Cobertura da lógica de negócio, clareza dos casos de teste
Documentação	Clareza do README e facilidade para executar o projeto